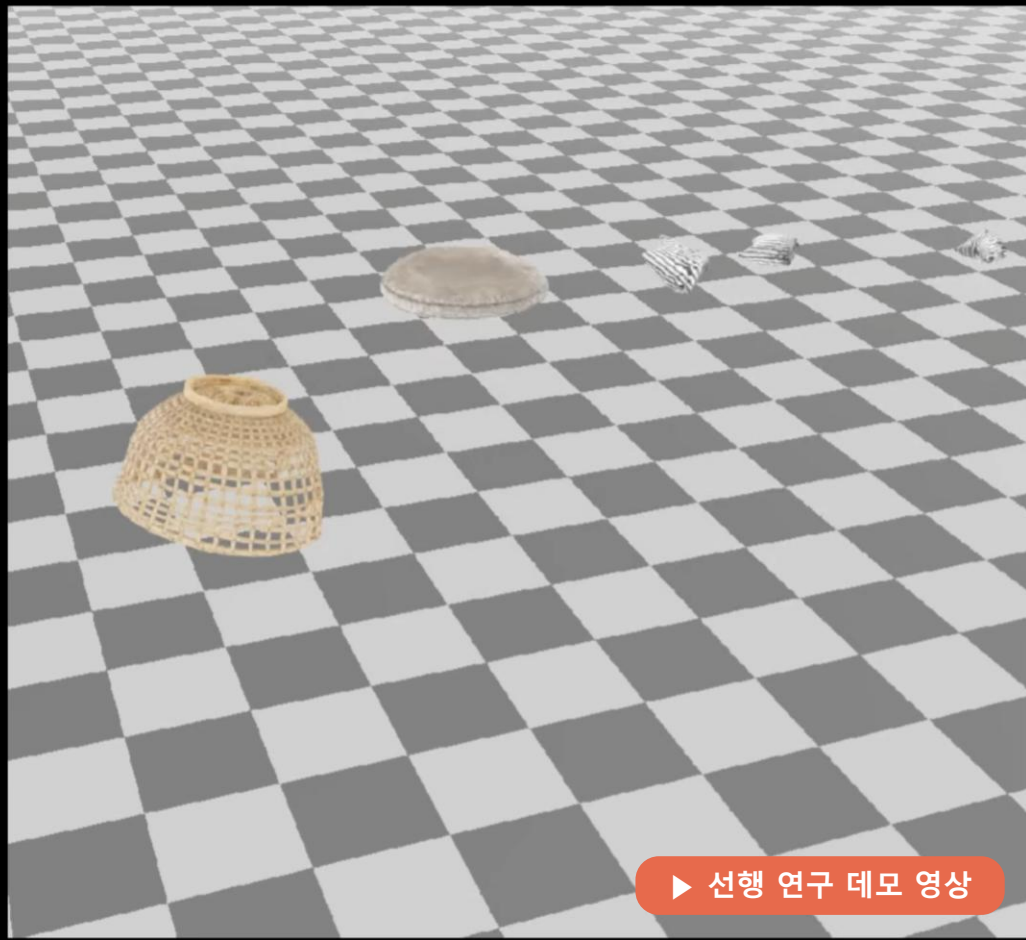




▶ 선행 연구 데모 영상

DIGITAL TWIN R&D PORTFOLIO

3DGS-to-Sim: Graph-Based Deformable Digital Twins for Robot Interaction

로봇이 만지고 변형하는 3DGS 디지털 트윈 · 가우시안 그래프 물리(APG-GS) × Isaac Sim

Keyword : 3DGS · OpenUSD · Isaac Sim 6.1(PhysX/Fabric) · CUDA XPBD DLL · Python(ctypes)

권상민 · Sangmin Kwon

영상·PPTX: <https://minn-k.github.io/3dgs-isaac-sim-xpbd/>

Isaac Sim 의 3DGS — 렌더링은 되지만 물리 객체는 아님

문제 로봇이 스캔 물체를 다루려면 보이는 모양대로 부딪히고, 눌리고, 힘을 주고받아야 함

필요 디지털 트윈에 필요한 것

1

보이는 모양대로 충돌

충돌 모양 = 화면에 그려진 모양

2

눌렀다가 돌아옴

힘을 받으면 모양이 바뀌고 복원

3

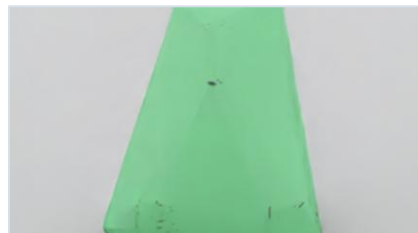
로봇과 힘을 주고받음

잡기·충돌이 양쪽으로 작용

현황 Isaac Sim 6.1 기본 지원



보이는 것 가우시안 11.4만 개



PhysX 가 계산 블록 겹질 1개

- 3DGS 가져오기 · RTX 렌더링 .ply · .spz → ParticleField3DGaussianSplat
- 가우시안 속성 실행 중 갱신 위치 · 방향 · 크기 (Fabric)
- ✗ ParticleField ≠ PhysX 충돌체 CollisionAPI 를 붙여도 충돌 모양 없음 (실측)
- △ 별도 충돌 프록시 강체 메쉬 · 블록 · SDF — 프록시만 움직임
- ✗ 가우시안 변형은 계산 안 함 접촉 → 위치·회전·크기 연결 없음

해결 가우시안 물리 층 추가

Isaac Sim · PhysX
로봇 · 강체 (기본 기능)

접촉 · 충돌체 자세

APG-GS 가우시안 물리
그래프 · XPBD · 부피 · 타원체 접촉
기존 연구 엔진 → 이번 작업으로 Isaac 에 연결

위치 · 회전 · 크기

Fabric → RTX 렌더링
ParticleField (기본 기능)

- 1 그려진 타원체로 충돌
- 2 11만~20만 개 매 프레임 변형
- 3 PhysX 강체와 충격량 교환

기대 효과

1

스캔 자산을 바로 시뮬레이션에

Gaussian 자산 측 별도 mesh 변환·수작업 collision proxy 없이

2

렌더링 공분산에 정합된 충돌 형상

잡기 · 충돌 결과가 화면과 일치 → 검증 · 데이터 생성

3

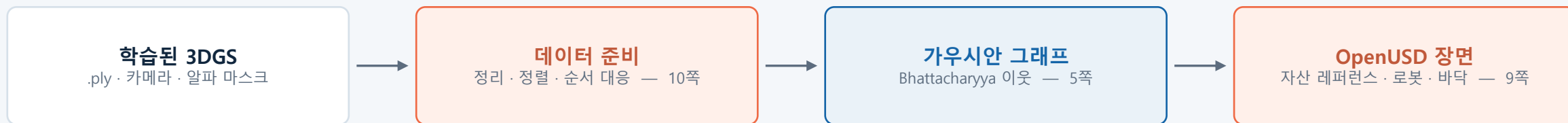
기존 Isaac 워크플로 그대로

OpenUSD 자산 · PhysX 로봇 · RTX 렌더와 호환

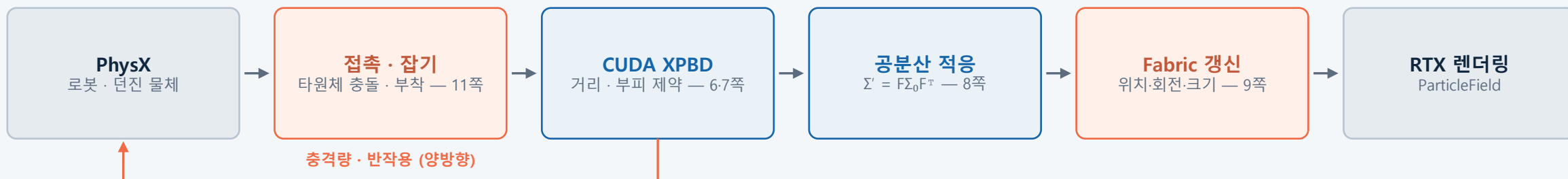
전체 프레임워크 — 3DGS 자산 · 가우시안 물리 · Isaac Sim

한 번 준비해 두고, 매 물리 스텝마다 PhysX 와 가우시안 물리가 같은 시각으로 돈다

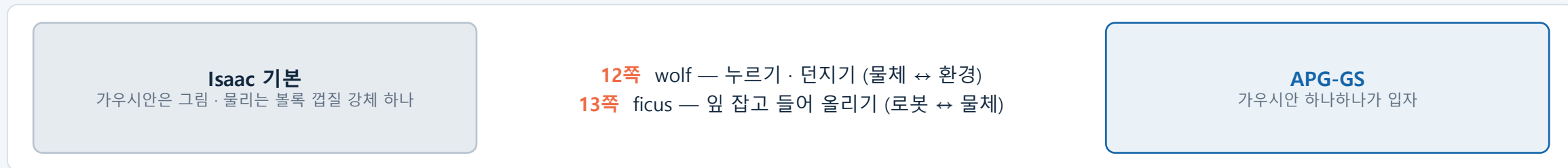
① 준비 한 번



② 실행 매 물리 스텝 1/60 s

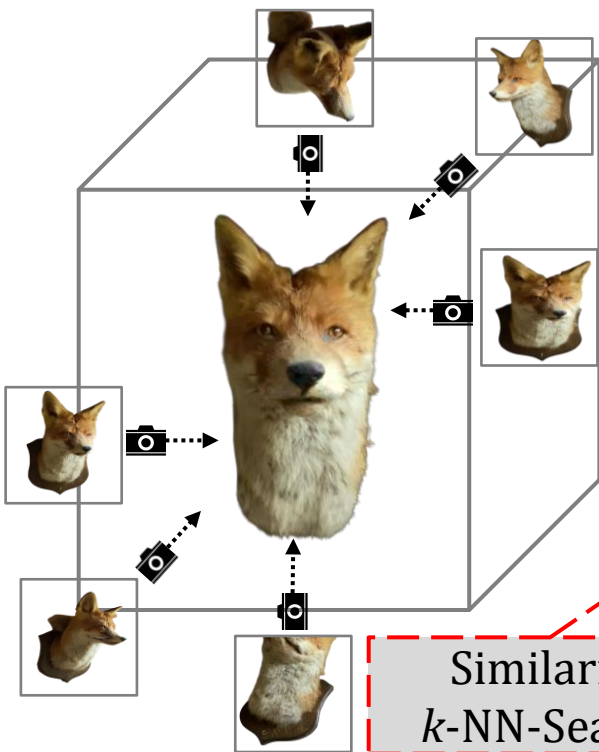


③ 검증 같은 입력으로 Isaac 기본 방식과 비교

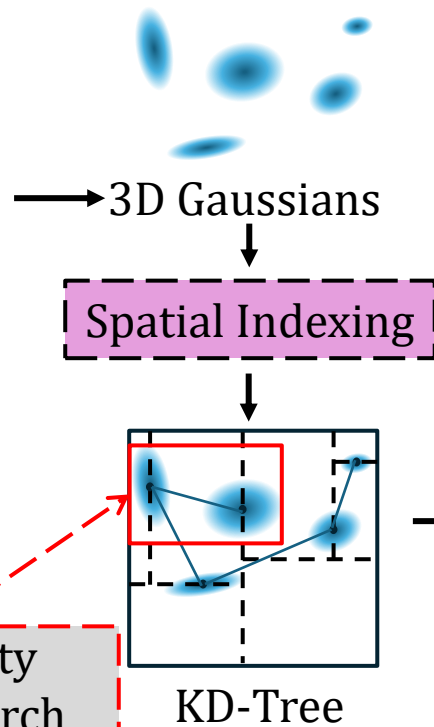


APG-GS — 메쉬 없는 3DGS 변형

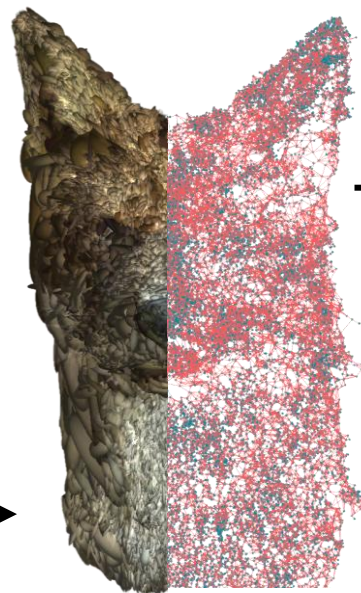
Multi-view Inputs



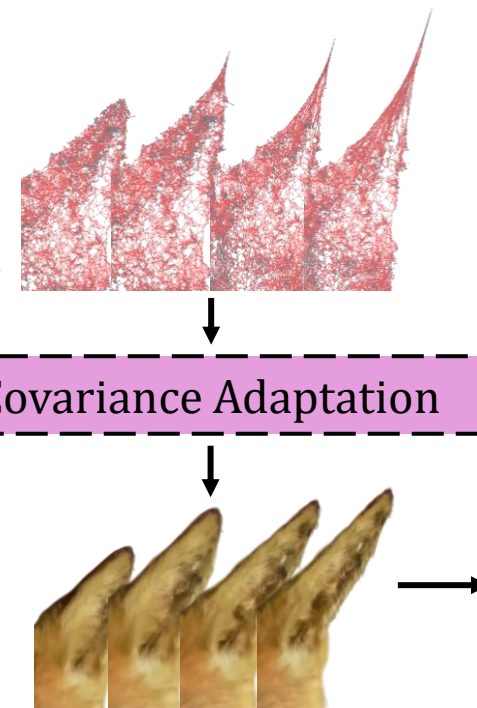
3DGS Reconstruction



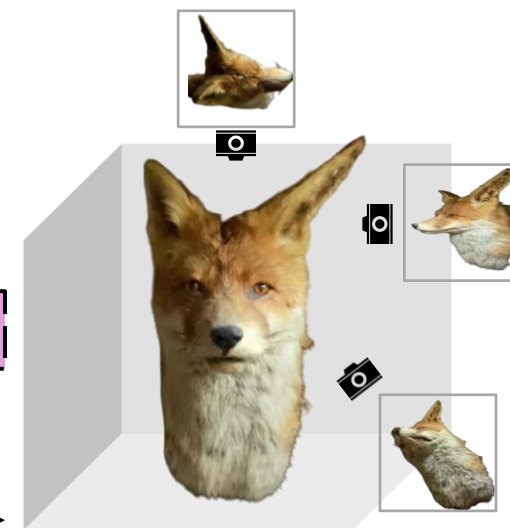
Similarity Graph Construction



Deformation



Real-time Splatting



01 그래프를 만든다

서로 붙어 있을 가능성이 높은 가우시안만 이웃으로 연결해 변형이 퍼질 경로를 만든다.

02 그래프 위에서 변형을 푼다

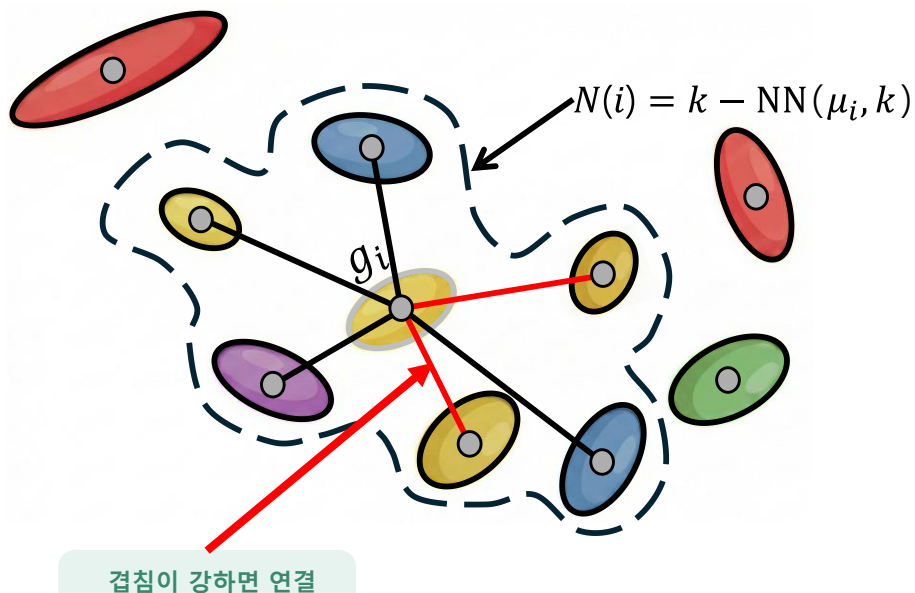
빠른 변형 전파에는 ChainMail, 제약 기반 복원에는 XPBD를 사용해 국소 입력이 전체 모양을 망가뜨리지 않게 한다.

03 렌더링 공분산도 함께 바꾼다

중심 위치만 옮기지 않고 이웃의 변형을 추정해 가우시안의 방향과 크기도 갱신한다.

Bhattacharyya 겹침 기반 가우시안 그래프

공분산 타원체의 겹침을 사용하면 거리·방향·크기를 따로 손으로 가중할 필요가 줄어든다.



후보 찾기

01

중심 거리 기준으로 가까운 후보를 준비

겹침 점수화

02

공분산 유사도와 Mahalanobis 항을 계산

연결 보존

03

강한 겹침만 남기고 비정상적으로 긴 연결 제외

Bhattacharyya distance

$$D_B(i, j) = \frac{1}{8} d_{ij}^T \bar{\Sigma}_{ij}^{-1} d_{ij} + \frac{1}{2} \ln \left(\frac{\det(\bar{\Sigma}_{ij})}{\sqrt{\det(\Sigma_i) \det(\Sigma_j)}} \right)$$

첫 항은 중심의 차이, 둘째 항은 공분산 모양의 차이를 함께 본다.

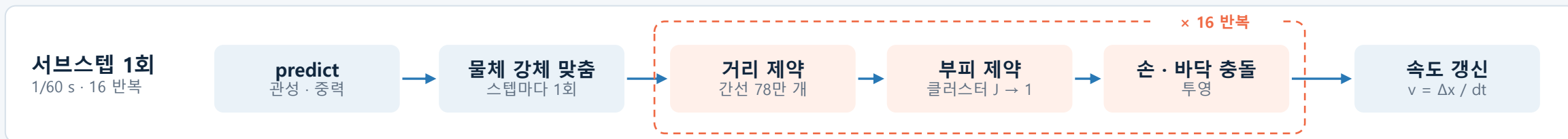
직관 확인

두 타원체가 같은 방향·크기라면 식의 첫 항은 "각 축 길이로 정규화한 거리", 즉 Mahalanobis 거리로 줄어듭니다.

$$\Sigma_i = \Sigma_j = \Sigma \text{ 이면 } D_B = \frac{1}{8} d^T \Sigma^{-1} d = \frac{1}{8} \cdot d_M^2$$

병렬 XPBD 솔버 — Jacobi · Gauss-Seidel 측정 비교

문제 10만 입자를 60 Hz 로 — 이웃끼리만 주고받는 제약은 물체 전체로 느리게 퍼짐



JACOBI

기본값

입자마다 1스레드 · 이웃 보정의 평균 · 버퍼 2개

$$\Delta\lambda = (-C - \alpha\tilde{\lambda}) / (w_i + w_j + \alpha), \quad \alpha\tilde{=} \alpha / dt^2$$

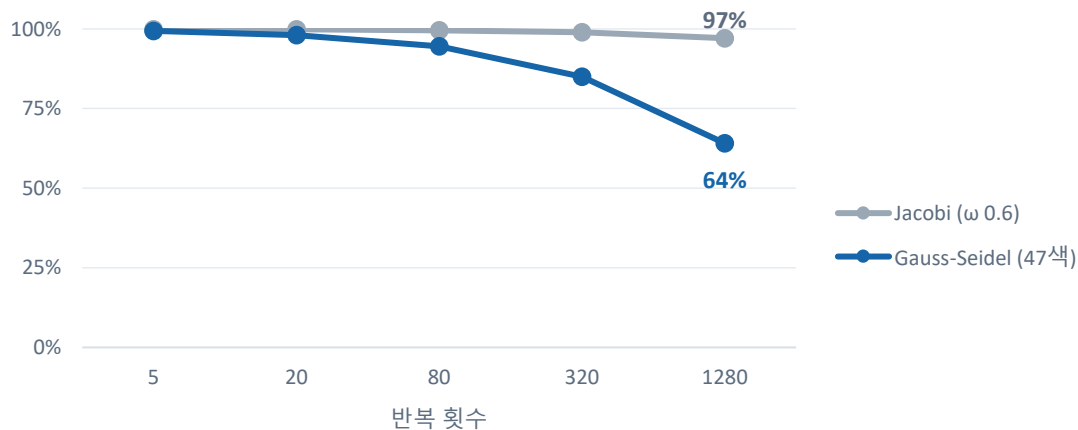
GAUSS-SEIDEL

토글

간선 47색 칠하기 · 색마다 커널 1번 · 제자리 갱신

$x_i \ += w_i \cdot \Delta\lambda \cdot n$, $x_j \ -= w_j \cdot \Delta\lambda \cdot n \rightarrow$ 다음 색이 바로 본다

남은 비틀림 오차 (초기 = 100%) · face 102,363점 · 거리 제약만



~15×

GS 수렴 속도
같은 오차까지 반복 수
ms 기준 ~6×

64%

GS 실험 1,280회 뒤
남은 비틀림 오차
Jacobi 97%

물체 전체 규모의 오차는 국소 솔버로 느리다

→ 물체 단위 강체 맞춤을 맨 위 층으로 두고, 국소 제약은 Jacobi
GS 는 정확도 이득이 없어 토글로 뒀음 (FEM 꺾적 오차 5.3 → 5.7%)

부피 제약 가속 — atomic 없는 gather · 클러스터당 1워프

문제 클러스터 하나에 멤버 수백 개 — 스레드당 처리는 느리고, atomic 누적은 결과가 매번 다름

BEFORE · SCATTER

클러스터가 멤버 칸마다 **atomicAdd** 로 더함

```
atomicAdd(&dp[m], w_m·Δλ·∇J_m) // 멤버 m 마다
경합 · 더하는 순서가 매번 달라 결과가 비결정적
```

AFTER · GATHER

클러스터는 자기 칸에 **coef·ctr· Σ^{-1}** 만 쓰고, 입자가 전치 CSR 로 읽어 합산

```
for c in clusters_of(i): dp_i += w_i·coef[c]· $\Sigma^{-1}[c]$ ·(x_i - ctr[c])
atomic 0 · 같은 입력 두 번 → 비트 단위 동일
```

WARP

클러스터 하나를 32스레드가 나눠 읽고 **shuffle** 로 합산

```
acc = warpAllSum(moments(x[member[lane::32]]))
멤버 위치 읽기 3회 → 1회 · 입자 쪽 부하 불균형도 제거
```

기호

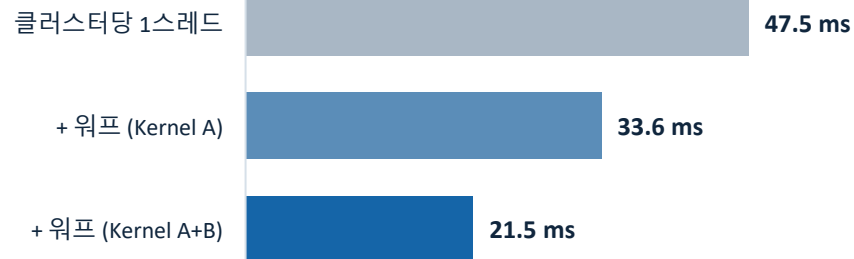
$J = \sqrt{(\det \Sigma / \det \Sigma_0)}$ 클러스터 부피비 (1 = 원래 부피). 제약 $C = J - 1 \rightarrow 0$ 으로
 Σ , ctr 클러스터 멤버 위치의 공분산·중심 (Σ^{-1} = 역행렬)
 $\nabla J_m = (J/n) \cdot \Sigma^{-1} (x_m - \text{ctr})$ 멤버 m 을 옮길 때 부피비가 커지는 방향
 $\Delta \lambda$ 이번 반복의 보정량 (XPBD 라그랑주 승수 증분), w = 역질량
 $\text{coef}[c] = \Delta \lambda \cdot J/n$ 클러스터마다 한 번 계산해 두는 계수 (n = 멤버 수)
 dp_i 입자 i 의 위치 보정 — 속한 클러스터들에서 받은 값의 평균

① 스레드 → 워프 (같은 설정)

1프레임 시간 · 짧을수록 빠름 · SIBR 뷰어, face 102,363점, 멤버 최대 256

2.21배

빨라짐 (47.5 → 21.5)



② 워프라서 쓸 수 있게 된 설정

20.0 → 5.7 ms

1프레임 · 175 FPS (Standalone APG-GS Engine / SIBR기준)

전 모든 가우시안이 클러스터 중심(10만 개), 멤버 64개까지만 (나머지 버림)
 후 중심 2,205개, 멤버 제한 없이 전부 — 클러스터가 적으면 스레드 방식은 GPU 가 거의 놀아(점유 1%) 워프가 전제 · 부피 오차 0.70 → 0.78% (비슷)

검증

계산 순서만 바뀌고 결과는 같음 — 스레드 ↔ 워프 부피비 J 평균 0.99070, 소수 5자리 일치
 물리적으로 맞는지 — FEM(Neo-Hookean) 대비 부피 오차 1% 이내 (합성 block test에서 Neo-Hookean FEM 대비 volume ratio error < 1%)

렌더링 공분산 적용 — $\Sigma' = F \Sigma_0 F^T$

문제 물리는 가우시안 중심만 옮김 — 방향·크기를 그대로 두면 늘어난 면이 찢어짐

변형 전



위치만 갱신 — 늘어난 면이 찢어짐



위치 + 공분산 갱신 — 면이 이어짐



deformedShapeKernel — 가우시안마다 1스레드

```
A      =  $\Sigma q \cdot p^T$  ( $\Sigma p \cdot p^T$ )-1    // p: rest 이웃, q: 현재 이웃
R, S = polar(A)                                // 회전 · 늘어남 분리
 $\Sigma' = F \Sigma_0 F^T$                         // 렌더러와 같은 식, 제한 없음
scale, quat = eig( $\Sigma'$ )                     // Splat prim 입력 형식
```

그림: lego 샵 끝 3 cm 를 8 cm 당김 · Isaac RTX 렌더

0.4 ms

공분산 적용 비용
프레임의 약 2%

PART 2 · Isaac 이식

화면 밖 가우시안까지 계산 · 같은 Σ' 를 충돌(11쪽)에 재사용

Isaac Sim 통합 — OpenUSD 장면 · PhysX 동기화 · Fabric 갱신

문제 연구 엔진은 자체 뷰어 안에서만 동작 — 표준 장면·물리 엔진·로봇과 연결 필요

USD 장면 구성

```
wolf_scene.usda      Z-up · 1 unit = 1 m
/World
├── Wolf / Splat      → ref wolf_splat.usd
│   └── ParticleField3DGaussianSplat
├── PhysicsScene · GroundPlane
├── RobotBase/Franka → ref Isaac 자산 서버
│   └── 손가락·손바닥 대리체 xpb:collider
└── Thrown/obj_*      Cube·Sphere + RigidBody
```

레퍼런스 구조 — 자산 경로만 바꾸면 다섯 자산에 같은 스크립트

RUNTIME — Python(ctypes) ↔ CUDA DLL ↔ PhysX · Fabric

```
physx.subscribe_physics_on_step_events(on_step)
def on_step(dt):
    # PhysX 스텝마다 1/60 s
    xpb.set_colliders(pads_at(hand))
    xpb.set_attached(held, hand @ local)
    xpb.step(dt)
def on_update(e):
    # 렌더 프레임마다
    P, q, s = xpb.read_shapes()
    rt_pos.Set(P); rt_rot.Set(q); rt_sc.Set(s) # Fabric
```

최적화 병목 = 렌더러가 바뀐 Splat 을 매 프레임 다시 올리는 일 (프레임의 45%) — 그 횟수를 줄였다

렌더 1프레임 = 물리 4스텝

재적재 비용을 4스텝이 나눠 냄

0.16x → 0.49x

시뮬레이션 속도 (실시간 대비) · 3.1배

화면 9.3 → 7.3 FPS

안 변하면 Splat 을 쓰지 않음

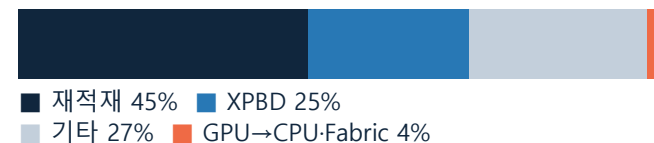
이동 < 임계값이면 Fabric 쓰기 생략

6.9 → 12.6 FPS

물체가 멈춘 동안 · 1.8배

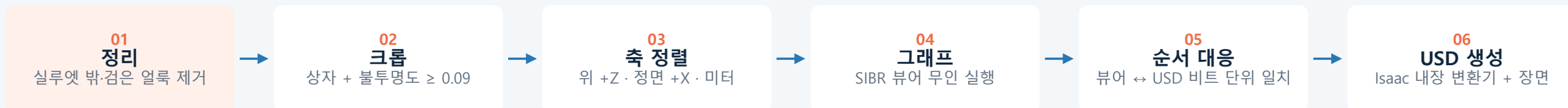
ficus 10만 점 · 1280×720

프레임 146 ms 분해

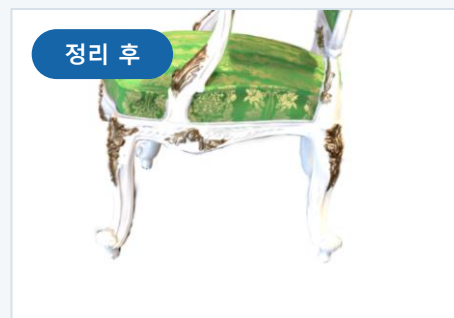
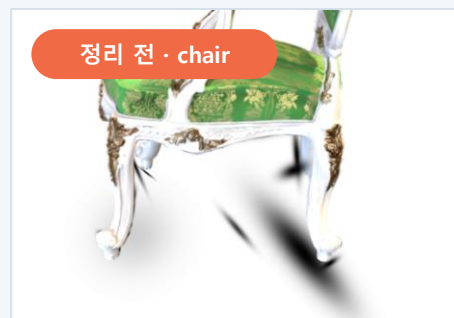
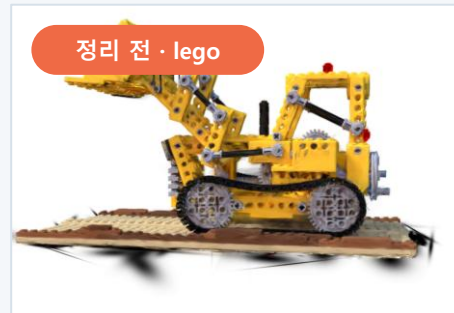


데이터 준비 — 학습된 3DGS → 시뮬레이션 자산 (한 명령)

문제 학습된 3DGS 는 축·크기가 제각각이고 floater 가 섞여, 그대로는 시뮬레이션에 쓸 수 없음



자산	출처	가우시안	정리 결과
face	실촬영 (인물)	102,363	기준 자산 — 누르기·집기·던지기 먼저 검증
wolf	실촬영	114,002	정면 방향 지정 (yaw +90°)
figus	NeRF synthetic	103,336	앞 쪽이 무거움 (가우시안 질량 균등)
chair	NeRF synthetic	169,883	floater 153개 제거 · 그래프 조각 6 → 1
lego	NeRF synthetic	199,167	floater 4,286개 제거 · 조각 7 → 1 · 잘못 등록된 시점 6/100 제외



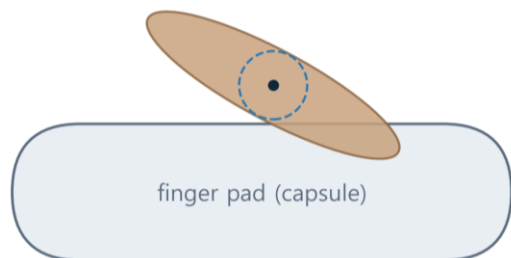
floater 정리 학습 카메라마다 투영해 실루엣 밖에 떨어지는 가우시안과 크고 어두운 얼룩을 지운다 — NeRF 합성 데이터는 검은 배경으로 학습돼 검은 floater 가 남고, Isaac 의 밝은 배경에서 드러난다

로봇 상호작용 — Gaussian 접촉 · 부착 기반 Grasp · 양방향 충격량 교환

문제 문제 Gaussian 중심점만으로 접촉을 판정하면 실제 렌더링 형상보다 충돌 영역이 작아져 로봇 손가락과 강체가 물체 내부로 침투

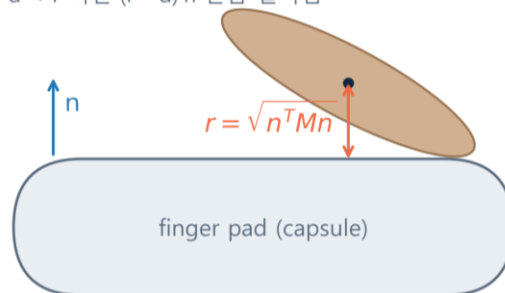
이전: 중심점 + 여유 8 mm

중심 + 고정 반경으로만 판정
→ 길쭉한 가우시안이 패드에 묻힘



현재: τ 등고면 타원체

$M = k^2 \Sigma'$, $k = \sqrt{2 \ln(\alpha/\tau)}$
 $d < r$ 이면 $(r - d) \cdot n$ 만큼 밀어냄



충돌 = 그려진 타원체

로봇 손(캡슐·상자) · 던진 물체 공통

35 → 1개

손가락 안으로 파고든 가우시안

face 를 손가락 캡슐(반지름 9.5 mm)로 3 cm 누를 때, 캡슐 안으로 1 mm 넘게 들어간 가우시안 수.
점으로 판정 35개 → 타원체로 판정 1개 — 적을수록 보이는 표면이 손가락을 뚫지 않음

잡기 = 부착

손가락 사이 가우시안을 손에 고정

0.17 → 0.29 m

들어 올린 물체의 무게중심 높이

face 를 한 팔로 들 때. 잡은 점의 강체 맞춤 무게 0 이면 잡힌 곳만 늘어나고 몸은 바닥에 (0.17 m),
나머지와 1:1 이면 몸 전체가 손을 따라 들림 (0.29 m) · 20 mm 넘게 늘어난 가우시안 2,106 → 8개

contactShapePack + kinColliderProject

```
k = sqrt(2 * ln(alpha/tau)); M = k^2 * Sigma' // 불투명도 tau=0.2 등고면
r = sqrt(n^T * M * n) // 충돌체 법선 방향 두께
if (d < r) x += (r - d) * n // 타원체 표면까지 밀어냄
Sigma' = 8쪽 공분산 재사용 → 추가 비용 0.08 ms/프레임
```

던진 물체 = 타원체 접촉 + 두 강체 충격량 (양방향)

- ① 매 스텝 가우시안 타원체를 밀어냄 → 맞은 곳이 눌림
- ② 빠르게 닿으면 $J = (1+e) \cdot v / (1/M_{\text{물체}} + 1/M_{\text{가우시안}} + \text{회전항})$
가우시안 몸 전체에 +J, PhysX 물체에 -J (2 스텝에 나뉨)
- ③ 얹혀 있으면 밀어낸 만큼의 반작용 $m \cdot \Delta x / dt$ 를 PhysX 에

한계 잡기는 마찰이 아닌 부착 · 바닥은 점 접촉

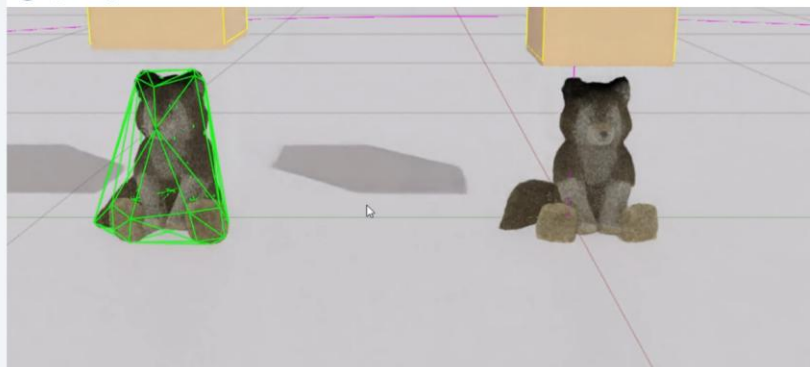
비교 1 · 물체 ↔ 환경 — 누르기 · 던지기 (wolf)

동일한 3DGS 자산에 동일한 낙하·압축·충돌 조건을 적용해, rigid collision proxy와 Gaussian physics layer의 상호작용 차이를 비교

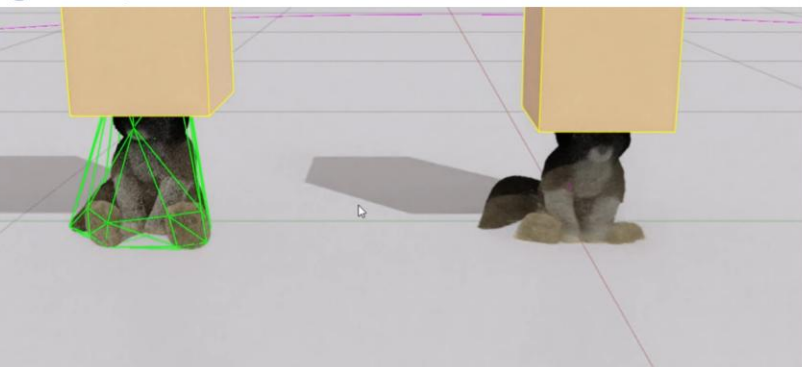
디지털 트윈 요소 → 이렇게 구현

왼쪽 · Isaac 기본 (블록 겹칠 강제) 오른쪽 · APG-GS XPBD

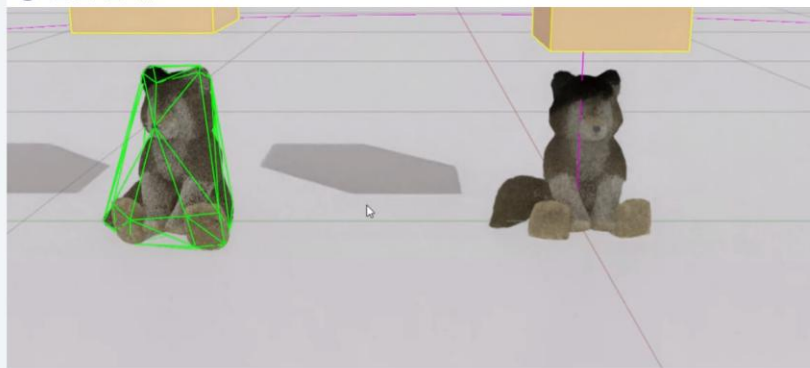
① 누르기 전 같은 높이에 선 두 wolf



② 판으로 누름 오른쪽 27.0 → 20.2 cm 눌림 · 왼쪽은 판이 겹칠에 겹침



③ 판을 댄 뒤 오른쪽 27.1 cm 로 복원 · 왼쪽 30.2 cm 그대로



④ 공 충돌 오른쪽 맞은 곳만 눌리고 공이 튕김 · 왼쪽 통째로 기울어짐



▶ 비교 1 영상 보기

1 렌더링 형상에 정합된 접촉

APG-GS 렌더링에 사용되는 Gaussian 공분산 타원체를 접촉 판정에도 사용

Isaac 기본 단일 convex hull 사용 — 다리 사이와 꼬리 아래의 빈 공간까지 하나의 충돌 영역으로 포함

2 국소 변형 후 형상 복원

APG-GS 높이 27.0 → 20.2 cm로 압축외력 제거 후 27.1 cm로 복원

Isaac 기본 rigid collision proxy는 변형되지 않아 시각적 3DGS와 충돌 형상 사이에 불일치 발생

3 국소 변형 + 양방향 충격 반응

APG-GS 충돌 지점은 국소적으로 변형되고, 계산된 반작용은 PhysX 물체에 다시 전달

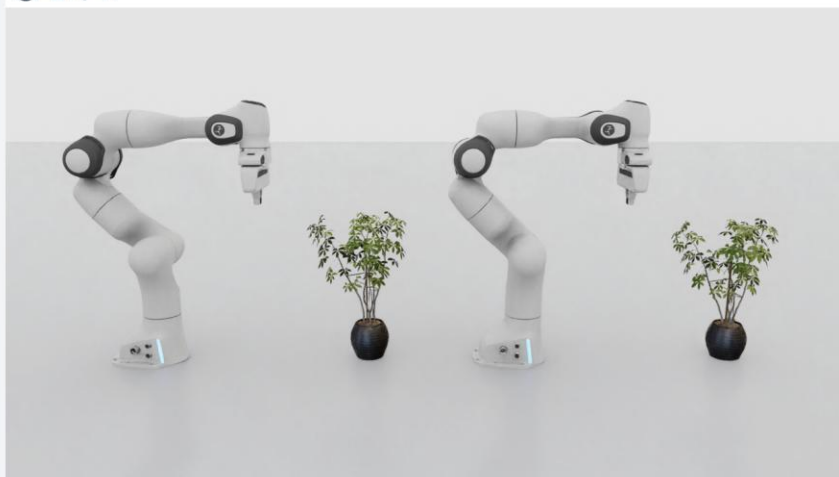
Isaac 기본 하나의 rigid body로 처리되어 물체 전체가 함께 이동·회전

비교 2 · 로봇 ↔ 물체 — 잎 잡고 들어 올리기 (ficus)

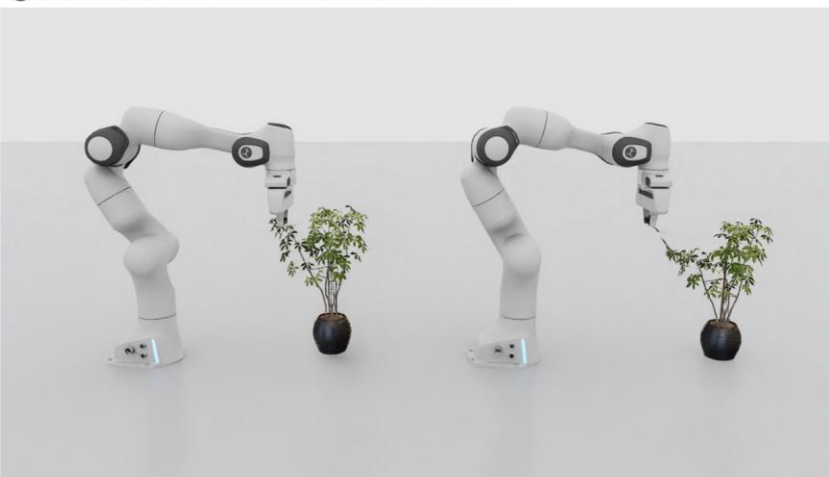
두 Franka 에 같은 관절 명령 — 가장 바깥쪽 위에서 집어 12 cm 들어 올리고, 1.5 s 들고 있다가 옆에 내려놓는다

왼쪽 · Isaac 기본 (블록 껍질 강체) 오른쪽 · APG-GS XPBD

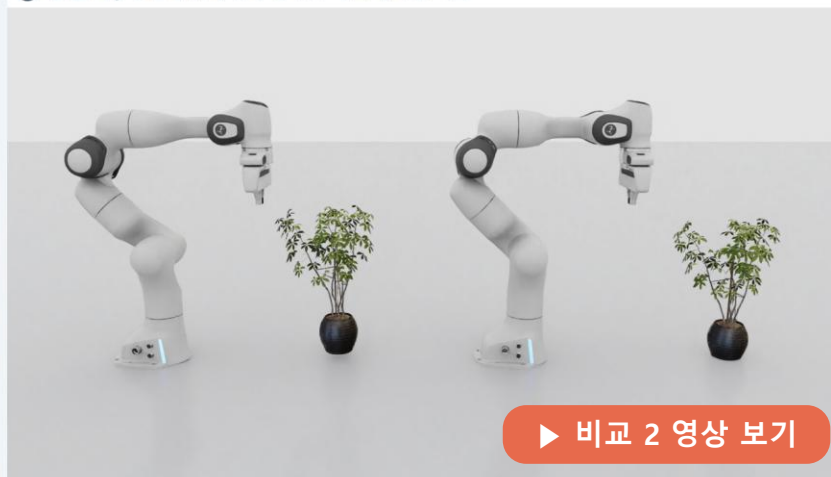
① 잡기 전



② 10 cm 당김 왼쪽 식물 전체가 끌려옴 · 오른쪽 가지만 힘



③ 놓은 뒤 오른쪽 0.6 s 안에 제자리 · 왼쪽 옮겨진 자리



▶ 비교 2 영상 보기

1 Gaussian 단위의 국소 grasp

APG-GS 손가락 사이의 실제 Gaussian 영역을 선택해 부착(가우시안 970개)

Isaac 기본 convex hull이 잎 사이 빈 공간까지 차지해 실제 보이는 부분과 grasp 영역이 일치하지 않음

2 하중에 따른 국소 변형·복원

APG-GS 잡힌 가지 주변이 하중에 따라 변형2배 이상 늘어난 연결은 0.14%, 내려놓으면 형상 복원

Isaac 기본 하나의 rigid body로 처리되어 부분적인 형상 변화 없음

3 물체의 자유로운 자세 반응

APG-GS 국소 grasp 지점만 고정되어 중력에 따라 무게중심이 손 아래로 이동하며 물체가 매달림

Isaac 기본 fixed joint가 상대 자세까지 고정해 원래 자세를 유지한 채 이동